

Bas — CM 3

Éléments de shell de Bourne : expressions régulières, scripts, variables, structures de contrôle ·
UE Bases de l'Architecture et des Systèmes · L1 Informatique, UPS · Semestre 1 2026-2027

1. Expressions régulières

Une **expression régulière** est un moyen de définir des modèles (*pattern*) de lignes de texte, transmis à des commandes. C'est une chaîne de caractères à laquelle une ligne de texte peut correspondre ou non, composée de **caractères normaux** et de **caractères spéciaux = métacaractères des expressions régulières**.

Les métacaractères des expressions régulières sont **différents** des métacaractères du shell (vus au CM précédent) : ce sont deux systèmes de caractères spéciaux distincts, qui s'appliquent à des moments différents. Les expressions régulières ne sont pas spécifiques au shell de Bourne : très utilisées pour l'écriture de scripts shell, souvent au travers de commandes comme `expr` et surtout `grep`.

Principaux métacaractères des expressions régulières

Métacar.	Signification	Exemple
.	caractère quelconque.	<code>..</code> → modèle d'une ligne formée de 2 caractères quelconques (comme <code>aa</code> , <code>bc</code> , <code>45</code> , <code>6t</code>).
*	répétition, du caractère qui précède, de longueur quelconque éventuellement vide.	<code>a*</code> → modèle d'une ligne formée d'un nombre quelconque de <code>a</code> (y compris 0).
^	début d'une ligne.	<code>^a.*</code> → modèle d'une ligne commençant par le caractère <code>a</code> .
\$	fin d'une ligne.	<code>.*a\$</code> → modèle d'une ligne se terminant par le caractère <code>a</code> .
\	protège le caractère suivant, normal ou métacaractère des expressions régulières.	<code>a\b</code> → modèle d'une ligne formée des 3 caractères <code>a</code> <code>\$</code> <code>b</code> .
\{n\}	répétition de n fois le caractère précédent, normal ou métacaractère.	<code>a\{3\}</code> → modèle d'une ligne formée de 3 caractères <code>a</code> .
[liste]	caractères de la liste, définis par énumération ou par intervalle.	<code>^[a1b2]</code> → modèle d'une ligne contenant un caractère <code>a</code> ou <code>b</code> ou <code>1</code> ou <code>2</code> .

Écriture de la chaîne

- **Entre apostrophes ' '** (recommandé) : les caractères `$`, `*`, `\`, etc. ne seront **pas interprétés** comme des métacaractères du shell. Exemple : `'*a ?'`.
- **Seule ou entre guillemets " "** : certains métacaractères du shell **seront interprétés**. Exemple : `$var` ou `"$var"`.

2. Script

Définition & exécution

Un **script** est un ensemble de commandes Unix rassemblées dans un fichier. Commentaires : ce qui suit `#`. Fichier, le plus souvent, d'extension `.sh` (exemple : `./nom_script.sh`).

Lancement d'un script :

- À l'aide de la commande `sh` : `$ sh nom_script.sh [paramètres_du_script...]`
- En rendant le script exécutable puis en l'exécutant comme une commande : `$ chmod u+x nom_script.sh` puis `$ ./nom_script.sh [paramètres_du_script...]`

Shells père / fils

Un script = une commande. Elle est exécutée dans le shell « courant » = le « **shell père** ». Lancer l'exécution d'un script → lancer un **nouveau shell**, dit « shell fils », de même nature que le shell père (`csh`, `ksh`, `sh`...). Il porte le nom du script lancé, et c'est dans ce shell fils que les commandes du script seront interprétées.

Possibilité de préciser le shell invoqué au début du script : `#!/bin/sh`.

Paramètres positionnels

Dix paramètres positionnels : `$0`, `$1`, ..., `$8` et `$9`. `$0` : nom du shell. `$1`, ..., `$9` : pas de valeur a priori (chaîne vide).

- **Attribuer des valeurs** à ces paramètres avec la commande `set`. Exemple : `set alpha beta` → `$1` vaudra `alpha` et `$2` vaudra `beta`.
- **Lancer le script avec des paramètres** : affectation selon l'ordre. Exemple : `./nom_script.sh 1 2 3` → `$0` vaudra `./nom_script.sh`, `$1` vaudra `1`, `$2` vaudra `2` et `$3` vaudra `3`.

3. Paramètres spéciaux

Leur nom commence par le métacaractère `$`. Leur valeur dépend exclusivement du contexte du shell.

Param.	Signification
<code>\$#</code>	nombre de paramètres.
<code>\$*</code> et <code>\$@</code>	chaîne de caractères composée par la liste des paramètres, séparés par des espaces. " <code>\$*</code> " est remplacé par un seul mot . " <code>\$@</code> " est remplacé par la liste des paramètres dans laquelle chaque paramètre est un mot différent .
<code>\$?</code>	code de retour de la dernière commande exécutée dans le shell. Après l'exécution d'un script, c'est le code de retour de la dernière commande exécutée par ce script.
<code>\$\$</code>	numéro du processus du shell.

Exemple — script `exemple.sh` :

```
#!/bin/sh
echo "\$0=$0, \$1=$1, \$2=$2, \$3=$3, \$4=$4, \
\$5=$5, \$6=$6, \$7=$7, \$8=$8, \$9=$9"
echo "\$#=$#, \$*=$*, \$?=$?, \$\$=$\$"
```

L'exécution de la commande `./exemple.sh toto titi tata` produit :

```
$0=./exemple.sh, $1=toto, $2=titi, $3=tata, $4=,
$5=, $6=, $7=, $8=, $9=
$#=3, $*=toto titi tata, $?=0, $$=19681
```

4. Variables et expressions

Tout shell permet de gérer des **variables**. Nom d'une variable composé de lettres, chiffres et du caractère `_` ; ne peut pas commencer par un chiffre (exemples : `var`, `ma_variable_13`, `temp31`). Valeur obtenue en faisant précéder son nom du caractère `$` (exemple : `$var` est la valeur de la variable `var`). La valeur d'une variable est toujours une **chaîne de caractères** (exemple : `123` qui est la chaîne `"123"`, ou `abc` qui est la chaîne `"abc"`).

Affectation

À l'aide de la commande `read` :

```
$ read var
hello
$ echo $var
hello
```

Ou en utilisant la syntaxe `variable=expression`, où `expression` est une expression du shell ayant comme valeur une chaîne de caractères (exemple : `var=hello`).

Attention : il n'y a **pas d'espaces** autour de `=`. La syntaxe dépend du shell (syntaxe différente en Cshell, par exemple).

Concaténation d'expressions

S'écrit en juxtaposant des expressions qui peuvent être :

- une chaîne de caractères (exemple : `toto`)
- la valeur d'un paramètre positionnel (exemple : `$2`)
- la valeur d'une variable (exemple : `$var`)
- la capture de la sortie standard d'une commande (exemple : ``pwd``)

En utilisant — si besoin est — les délimiteurs `"` et `'` → tenir compte de leur influence sur l'interprétation des métacaractères du shell (voir CM précédent).

```
var=hello
var="$var world !" # -> var vaudra "hello world !"
```

Variables prédéfinies du shell

Variable	Rôle
SHELL	nom du shell courant.
PS1	configuration de l'invite de commande (prompt).
HOSTNAME	nom de la machine.
USER	nom de l'utilisateur connecté (équivalent à LOGNAME).
HOME	répertoire d'accueil.
PWD	répertoire de travail courant.
PATH	liste de répertoires où rechercher les commandes.

(liste non exhaustive) La commande `env` permet d'afficher la valeur de toutes les variables d'environnement.

Variables vs paramètres positionnels

Ce sont **deux concepts différents** :

Pour modifier...	Comment
la valeur d'un paramètre	appeler le script avec des paramètres, ou utiliser la commande <code>set</code>
la valeur d'une variable	affecter une valeur à une variable, ou utiliser la commande <code>read</code>

Accès à la valeur — lever les ambiguïtés avec les accolades

On accède à la valeur d'une variable en faisant précéder son nom de `$`. On utilise des **accolades** `${...}` pour lever des ambiguïtés sur les limites du nom de variable.

Exemple 1	Résultat
-----------	----------

\$ vari=impecc \$ variable=parfait \$ echo \$variable	parfait
\$ echo \${vari}able	impeccable

Sans accolades, le shell prend le **nom de variable le plus long possible** après le \$: `$variable` désigne la variable `variable` en entier (ici `parfait`), jamais `vari` suivi du texte `able`. Avec `${vari}able`, les accolades délimitent explicitement le nom de variable `vari`, et `able` est collé juste après sa valeur substituée (`impecc + able = impeccable`).

Deuxième exemple, plus subtil — sur la même session (`var=p`) :

Commande tapée	Ce qu'il se passe
\$ var=p	la variable <code>var</code> vaut <code>p</code> .
\$ \$varpwd	le shell cherche la variable <code>varpwd</code> (et non <code>var</code> suivi de <code>pwd</code>), qui n'existe pas : sa valeur est la chaîne vide, comme si l'utilisateur avait appuyé sur « entrée » (aucune commande exécutée).
\$ \${var}wd	les accolades délimitent le nom <code>var</code> (valeur <code>p</code>), suivi du texte <code>wd</code> → la ligne tapée devient la commande <code>pwd</code> , qui s'exécute et affiche le répertoire de travail courant, par exemple <code>/users/linfg/l1inf201</code> .

Ce deuxième exemple montre bien pourquoi les accolades sont utiles : `${var}wd` construit littéralement la chaîne `pwd` à partir de la valeur de `var` et l'exécute comme une commande — sans accolades, le shell aurait cherché une variable nommée `varwd` au lieu de construire ce mot.

5. Structures de contrôle

Contrôlent l'ordre dans lequel les différentes commandes d'un script sont exécutées. Sont de **3 types** :

- **Séquence** : par défaut, les commandes s'exécutent l'une après l'autre dans l'ordre de leur écriture.
- **Choix / Sélection / Alternative ...** : permet de choisir d'exécuter un bloc de commandes plutôt qu'un autre.
- **Boucle / Répétition** : permet de répéter l'exécution d'un bloc de commandes.

Conditions

Un choix ou une répétition se base sur un **test** = une **condition**. Une condition est réalisée par l'appel d'une commande retournant un code : valant `0` → la condition est « vraie » ; différent de `0` → la condition est « fausse ». La commande peut être quelconque ; c'est souvent la commande `test` qui est utilisée.

Choix

if / elif / else	case
<pre>if <u>condition1</u> then séquence_commandes1 elif <u>condition2</u> then séquence_commandes2 else séquence_commandes3 fi</pre>	<pre>case <u>expression</u> in cas1) séquence_commandes1 ;; cas2 cas3) séquence_commandes2 ;; *) séquence_commandes3 esac</pre>

- `cas1`, `cas2`, `cas3`, etc. sont des **chaînes de caractères**, pouvant utiliser éventuellement les métacaractères du shell.
- Le caractère `|` signifie « ou » (ex. `cas2|cas3`) ci-dessus).
- Le caractère `*` (métacaractère du shell) signifie « n'importe quelle chaîne de caractères ne commençant pas par `.` » — c'est le cas par défaut classique en fin de **case**.
- Le double point-virgule `;;` équivaut à la commande **break** (il termine le bloc du **case** en cours).

Répétitions

while	until
<pre>while <u>condition</u> do séquence_commandes done</pre>	<pre>until <u>condition</u> do séquence_commandes done</pre>

```
for variable in liste_de_cas
do
    séquence_commandes
done
```

- `liste_cas` est une liste de **chaînes de caractères**, pouvant utiliser éventuellement les métacaractères du shell.
- La commande **break** peut être utilisée (dans **while**, **until** ou **for**).

La syntaxe `for i / do / ...` (sans `in liste_de_cas`) équivaut à `for i in "$@" / do / ...` : la variable `i` prend alors successivement la valeur de **chacun des paramètres positionnels** ayant reçu une valeur.

À retenir

- **Expressions régulières** (avec `grep/expr`) : `.` (car. quelconque), `*` (répétition 0+ du précédent), `^/$` (début/fin de ligne), `[liste]`, `\{n\}` — **distinctes** des métacaractères du shell ; écrire la chaîne entre apostrophes pour ne pas les laisser interpréter par le shell.
- **Script** : `sh script.sh` ou `chmod u+x + ./script.sh` ; s'exécute dans un **shell fils**. `$0..$9` = paramètres positionnels (`set` pour les fixer manuellement).
- **Paramètres spéciaux** : `$#` (nombre), `*/$@` (tous, en 1 mot / en mots séparés), `$?` (code retour), `$$` (PID du shell).
- **Variables** : `variable=expression` (**sans espace** autour de `=`) ou `read var` ; toujours des chaînes de caractères. Utiliser `${nom}suite` dès qu'un texte est collé juste après le nom de variable, sinon le shell l'inclut dans le nom (piège classique, voir l'exemple `${var}wd`).
- **Structures de contrôle** : `if/elif/else/fi`, `case/esac` (avec `|=ou`, `;;=break`), `while/until/for...do...done` ; une condition est le **code de retour** d'une commande (0 = vraie), souvent via `test`.